

Anexo - Errores cometidos y lecciones aprendidas



”Porque el club es de los socios, y la gestión es de SocioUnido”

Plataforma inteligente que centraliza la gestión administrativa de los clubes, optimizando el uso de espacios compartidos, previniendo la morosidad y asegurando los accesos al estadio.

Alumno	Correo electrónico	Padrón
Ascencio, Felipe Santino	fascencio@fi.uba.ar	110675
Ghosn, Lautaro Gabriel	lghosn@fi.uba.ar	106998
Guerrero, Martín	mguerrero@fi.uba.ar	107774
Zielonka, Axel	azielonka@fi.uba.ar	110310

Jefe de Cátedra Mg. Ing. Carlos Fontela cfontela@fi.uba.ar

Tutor Mg. Ing. Gabriel Piñeiro gpineiro@fi.uba.ar

Cotutor Ing. Agustín Perrotta aperrotta@fi.uba.ar

Índice

1. Anexo - Errores cometidos y lecciones aprendidas	3
1.1. Riesgos previstos que se materializaron	3
1.2. Riesgos no previstos y errores de estimación	3
1.3. Aciertos y lecciones positivas	4

1. Anexo - Errores cometidos y lecciones aprendidas

El desarrollo de **SocioUnido**, ejecutado a lo largo de 13 *sprints* bajo el marco de trabajo ágil *Scrum*, permitió una evolución continua del producto. Sin embargo, este proceso iterativo no estuvo exento de obstáculos. La identificación, gestión y resolución de estos desvíos constituyeron una fuente invaluable de madurez técnica y metodológica para el equipo. A continuación, se detallan los riesgos materializados, los errores de planificación y las lecciones positivas extraídas durante el ciclo de vida del proyecto.

1.1. Riesgos previstos que se materializaron

1. Curva de aprendizaje y concurrencia asíncrona

- **Contexto:** Se preveía que la adopción de ciertos componentes del *stack* tecnológico requeriría tiempo de adaptación.
- **Materialización:** La implementación de procesos asíncronos en FastAPI para manejar simultáneamente los *webhooks* de Mercado Pago y las peticiones del *bot* de Telegram generó cuellos de botella en la base de datos por el mal manejo de las sesiones (conexiones no liberadas).
- **Lección:** Se evidenció la necesidad de realizar pruebas de carga y estrés en etapas más tempranas. Se estandarizó el uso de gestores de contexto asíncronos para la base de datos, resolviendo el bloqueo y mejorando la estabilidad.

2. Dificultades en la migración de datos heredados

- **Contexto:** Era sabido que los clubes manejaban su información preexistente con baja formalidad.
- **Materialización:** Al simular la carga masiva del padrón de socios, el equipo se encontró con datos severamente corruptos (fechas inválidas, correos duplicados, documentos incompletos) que rompían las reglas de integridad de la base de datos relacional.
- **Lección:** Nunca se debe confiar en la pureza de los datos externos. Como contramedida, se desarrolló un *script* de sanitización e importación tolerante a fallos, capaz de limpiar la información e informar los registros omitidos sin detener todo el proceso de migración.

1.2. Riesgos no previstos y errores de estimación

1. Subestimación en el entrenamiento del modelo de lenguaje (PLN)

- **El error:** Se estimó que la integración del *bot* conversacional en Telegram tomaría un único *sprint*, asumiendo que el procesamiento de lenguaje natural comprendería fácilmente las consultas de los usuarios.
- **El impacto:** En las pruebas de campo, el modelo fallaba constantemente al intentar procesar el argot local, abreviaturas o el tono informal propios del ser humano. Esto obligó a dedicar un *sprint* adicional exclusivamente a reentrenar y refinar el reconocimiento de intenciones.
- **Lección:** La interacción humano-computadora requiere un margen de tolerancia mucho mayor en la planificación. Las interfaces conversacionales deben ser probadas con usuarios reales (no solo por desarrolladores) desde su concepción.

2. Desvíos en el diseño responsivo del *dashboard*

- **El error:** El panel administrativo fue diseñado inicialmente bajo el supuesto de que sería utilizado exclusivamente desde computadoras de escritorio en las oficinas de administración del club.
- **El impacto:** Durante la validación temprana, se reveló que muchos usuarios operan en movimiento desde tabletas o teléfonos celulares. Hubo que refactorizar gran parte de las vistas web en el quinto *sprint* para garantizar una experiencia verdaderamente responsiva.
- **Lección:** Nunca se debe asumir el contexto de uso de un usuario sin validarlo previamente. La estrategia *mobile-first* (móvil primero) debe aplicarse a todas las interfaces, incluso a las de carácter administrativo o de uso interno.

1.3. Aciertos y lecciones positivas

1. Adopción estricta de la arquitectura de microservicios

La decisión de dividir el sistema en dominios independientes (club, pagos, *bot*, accesos, analítica) demostró ser un acierto absoluto. Cuando el servicio de pagos presentó problemas temporales de latencia en el entorno de pruebas, el resto de la plataforma (como la reserva de espacios o el ingreso al estadio) continuó funcionando sin interrupciones. Esta tolerancia a fallos validó empíricamente la solidez de la arquitectura elegida.

2. Automatización temprana (CI/CD)

Configurar las tuberías de integración y despliegue continuo (junto con las herramientas de análisis de código estático) desde el segundo *sprint* representó una inversión de tiempo que se recuperó con creces. Esta práctica eliminó por completo los problemas de compatibilidad manual, asegurando que la rama principal de código siempre fuera desplegable y estuviera auditada contra errores críticos antes de llegar a producción.

3. La comunicación como núcleo del trabajo ágil

El cumplimiento estricto de las ceremonias de *Scrum* (especialmente las retrospectivas) permitió que el equipo sincerara sus bloqueos técnicos a tiempo. La rotación de tareas y la asistencia cruzada entre los responsables de *front-end* y *back-end* fomentaron un conocimiento integral del ecosistema, demostrando que la cohesión humana y la transparencia son tan importantes como la excelencia técnica para el éxito del proyecto.

4. QA automatizado con OpenClaw para validación de seguridad

La integración de un agente basado en OpenClaw para la ejecución de aseguramiento de calidad (QA) automatizado representó una de las decisiones más rentables del proyecto. Este enfoque permitió realizar escaneos profundos de seguridad (identificando la exposición de credenciales o trazabilidad de errores), validación de prevención de fuga de datos y garantizar que los *endpoints* estuvieran estructuralmente listos para interactuar con otras inteligencias artificiales. Delegar esta carga de auditoría a una herramienta inteligente liberó tiempo crítico del equipo de desarrollo, elevando la madurez y la ciberseguridad del código a estándares profesionales.

5. Especialización y libertad en la distribución de roles

Definir roles sumamente específicos y otorgar plena libertad creativa y resolutive a cada integrante optimizó drásticamente los tiempos de desarrollo. Al centralizar responsabilidades (como concentrar la documentación y el aseguramiento de calidad [QA] en un perfil, la arquitectura en otro, y el diseño UX/UI en el *front-end*), se eliminaron los cuellos de botella en la toma de decisiones. Esta autonomía permitió entregar un producto sumamente pulido en todos sus aspectos: desde un *back-end* seguro y escalable, pasando por una interfaz de usuario atractiva, hasta una presentación comercial y académica del más alto nivel.